数値計算法 第1回

数の表現

濱本 雄治 [1]

情報工学部 情報通信工学科

2026 年 4 月 13 日

[1] hamamoto@c.oka-pu.ac.jp

第1回の目標

1. 2 進数と 10 進数の関係を理解する
   - 2 進整数 vs. 10 進整数
   - 2 進小数 vs. 10 進小数
2. 浮動小数点数の原理を理解する
   - 指数部、仮数部、符号部
   - 最大値と最小値

整数の 2 進数表現

- 10 進数の整数 $(x)_{10}$ を $2^0, 2^1, 2^2, \cdots$ で展開して係数を並べる

$$
\begin{aligned}
(x)_{10} &= b_{n-1} \cdot 2^{n-1} + b_{n-2} \cdot 2^{n-2} + \cdots + b_0 \cdot 2^0 \\
&= (b_{n-1} b_{n-2} \cdots b_0)_2
\end{aligned}
$$

  - 2^m の位の数は $b_m = 0$ or 1
  - 0 or 1 を n 個並べた 2 進数の情報量は n ビット

- c.f. 10 進数の整数は $10^0, 10^1, 10^2, \cdots$ の展開係数を並べたもの

$$
\begin{aligned}
(x)_{10} &= d_{n-1} \cdot 10^{n-1} + d_{n-2} \cdot 10^{n-2} + \cdots + d_0 \cdot 10^0 \\
&= (d_{n-1} d_{n-2} \cdots d_0)_{10}
\end{aligned}
$$

2 進整数を 10 進数に変換

方針

右から n 桁目の数字 (0 or 1) に 2^{n-1} を掛けてすべて足す

- 例: 2 進整数 00100101 を 10 進数で表す

$$1 \cdot 2^5 + 1 \cdot 2^2 + 1 \cdot 2^0 = 32 + 4 + 1 = 37$$

- 0 or 1 を n 個並べた 2 進数の情報量は n ビット

10 進整数を 2 進数に変換

方針

2 で次々に割って出てきた余りを右から並べる

- 例: 10 進数 37 を 2 進数で表す

$$
\begin{array}{rrcll}
2 &) & 37 & & \\
\hline
2 &) & 18 & \cdots & 1 \quad \leftarrow 2^0 \text{の桁} \\
\hline
2 &) & 9 & \cdots & 0 \quad \leftarrow 2^1 \text{の桁} \\
\hline
2 &) & 4 & \cdots & 1 \quad \leftarrow 2^2 \text{の桁} \\
\hline
2 &) & 2 & \cdots & 0 \quad \leftarrow 2^3 \text{の桁} \\
\hline
2 &) & 1 & \cdots & 0 \quad \leftarrow 2^4 \text{の桁} \\
\hline
 & & 0 & \cdots & 1 \quad \leftarrow 2^5 \text{の桁}
\end{array}
$$

余りを順に右から並べると $(37)_{10} = (100101)_2$

10 進整数を 2 進数に変換

- なぜ上の手順でうまくいくのか？
- $(x)_{10} = \underbrace{b_{n-1} \cdot 2^{n-1} + b_{n-2} \cdot 2^{n-2} + \cdots}_{2\text{ の倍数}} + \underbrace{b_0 \cdot 2^0}_{0 \text{ or } 1}$ を 2 で次々に割る

$$(x)_{10} \div 2 = \overbrace{\underbrace{b_{n-1} \cdot 2^{n-2} + b_{n-2} \cdot 2^{n-3} + \cdots}_{2\text{ の倍数}} + \underbrace{b_1 \cdot 2^0}_{0 \text{ or } 1}}^{\text{商 (1)}} \quad \overset{\text{あまり}}{\cdots} \quad b_0$$

$$\text{商 (1)} \div 2 = \overbrace{\underbrace{b_{n-1} \cdot 2^{n-3} + b_{n-2} \cdot 2^{n-4} + \cdots}_{\text{商 (2)} \leftarrow 2\text{ の倍数}} + \underbrace{b_2 \cdot 2^0}_{0 \text{ or } 1}}^{\text{商 (2)}} \quad \overset{\text{あまり}}{\cdots} \quad b_1$$

$$\vdots$$

$$\text{商 (n}-1) \div 2 = 0 \qquad \overset{\text{あまり}}{\cdots} \quad b_{n-1}$$

低い方の桁の数字から順に余りとして出てきた

小数の 2 進数表現

- 10 進数の小数 $(x)_{10}$ を $2^{-1}, 2^{-2}, 2^{-3}, \cdots$ で展開して係数を並べる

$$
\begin{aligned}
(x)_{10} &= b_{-1} \cdot 2^{-1} + b_{-2} \cdot 2^{-2} + \cdots + b_{-n} \cdot 2^{-n} \\
&= (0.b_{-1}b_{-2}\cdots b_{-n})_2
\end{aligned}
$$

  - 小数点以下 m 桁目の数は $b_{-m} = 0$ or 1

- c.f. 10 進数の小数は $10^{-1}, 10^{-2}, \cdots$ の展開係数を並べたもの

$$
\begin{aligned}
(x)_{10} &= d_{-1} \cdot 10^{-1} + d_{-2} \cdot 10^{-2} + \cdots + d_{-n} \cdot 10^{-n} \\
&= (0.d_{-1}d_{-2}\cdots d_{-n})_{10}
\end{aligned}
$$

2 進小数を 10 進数に変換

方針

小数点以下 n 桁目の数字 (0 or 1) に 2^{-n} を掛けてすべて足す

- 例: 2 進小数 0.1101 を 10 進数で表す

$$1 \cdot 2^{-1} + 1 \cdot 2^{-2} + 1 \cdot 2^{-4} = 0.5 + 0.25 + 0.0625 = 0.8125$$

10 進小数を 2 進数に変換

方針

2 を次々に掛けて出てきた整数部を小数点以下に左から並べる

- 例: 10 進小数 0.8125 を 2 進数で表す

$$
\begin{array}{lcll}
0.8125 \times 2 & = & 0.625 + 1 & \leftarrow 2^{-1}\text{の桁} \\
0.625 \times 2 & = & 0.25 + 1 & \leftarrow 2^{-2}\text{の桁} \\
0.25 \times 2 & = & 0.5 + 0 & \leftarrow 2^{-3}\text{の桁} \\
0.5 \times 2 & = & 0 + 1 & \leftarrow 2^{-4}\text{の桁}
\end{array}
$$

整数部を小数点以下に左から並べると $(0.8125)_{10} = (0.1101)_2$

10 進小数を 2 進数に変換

- なぜ上の手順でうまくいくのか？
- $(x)_{10} = b_{-1} \cdot 2^{-1} + b_{-2} \cdot 2^{-2} + \cdots + b_{-n} \cdot 2^{-n}$ に 2 を次々に掛ける

$$(x)_{10} \times 2 = \underbrace{b_{-1} \cdot 2^0}_{\text{整数部 0 or 1}} + \underbrace{b_{-2} \cdot 2^{-1} + \cdots + b_{-n} \cdot 2^{-n+1}}_{\text{小数部 (1)}}$$

$$\text{小数部 (1)} \times 2 = \underbrace{b_{-2} \cdot 2^0}_{\text{整数部 0 or 1}} + \underbrace{b_{-3} \cdot 2^{-1} + \cdots + b_{-n} \cdot 2^{-n+2}}_{\text{小数部 (2)}}$$

$$\vdots$$

$$\text{小数部 (n}-1) \times 2 = \underbrace{b_{-n} \cdot 2^0}_{\text{整数部 0 or 1}}$$

小数点以下の高い方の桁の数字から順に整数部として出てきた

前半のまとめ (2 進数と 10 進数の変換)

2 進整数 $\rightarrow$ 10 進数

右から n 桁目の数字に 2^{n-1} を掛けてすべて足す

10 進整数 $\rightarrow$ 2 進数

2 で次々に割って出てきた余りを右から並べる

2 進小数 $\rightarrow$ 10 進数

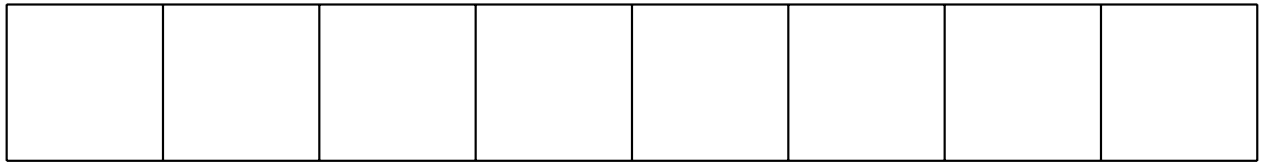
小数点以下 n 桁目の数字に 2^{-n} を掛けてすべて足す

10 進小数 $\rightarrow$ 2 進数

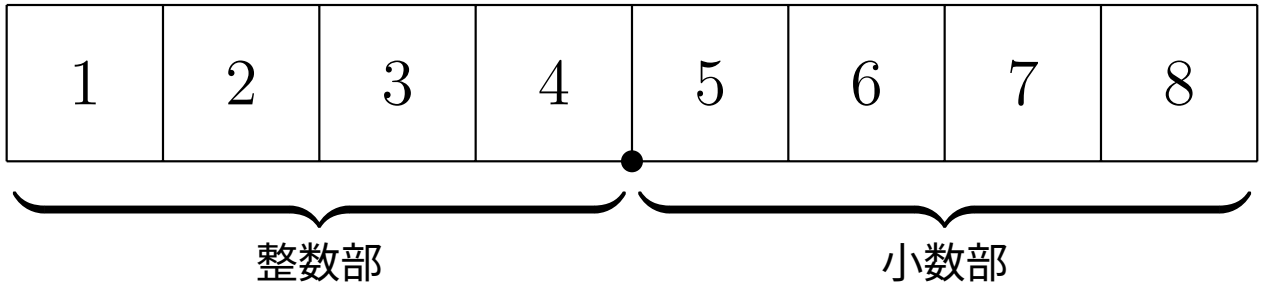
2 を次々に掛けて出てきた整数部を小数点以下に左から並べる

固定小数点数

- 8 桁の 10 進数で数値を表現する仮想コンピュータ (精度 8 桁)



- 仮に 4 桁目と 5 桁目の間に小数点を固定すると 1234.5678 は



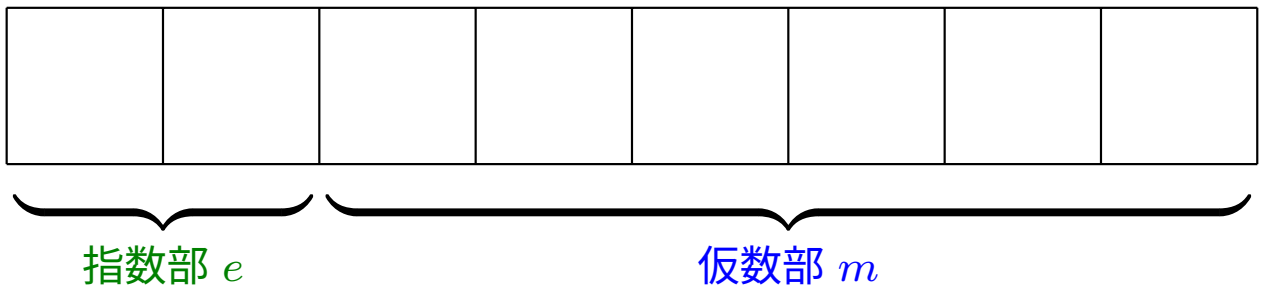
- この固定小数点数の問題
  - $9999.9999 \simeq 10^{+4}$ より大きな小数が表現できない
  - $0.0001 = 10^{-4}$ より小さい小数が表現できない

浮動小数点数

- より広い小数を表現するために
  - 初めの 2 桁を小数点の位置に使う (指数部)
  - 残りの 6 桁を有効数字に使う (仮数部)
- 小数を次の形に変形する (正規化)

$$0.m \times 10^{e-50} \qquad (00 \leq e \leq 99)$$

  - m は 6 つの数字の並びを表し、一番左の数字は 0 でない
  - e の後のバイアス -50 により、指数は $-50 \leq e - 50 \leq 45$ の範囲



浮動小数点数の例

- 例えば 1234.5678 を 8 桁 10 進の浮動小数点数で表すと

$$1234.5678 = 0.12345678 \times 10^4 = 0.12345678 \times 10^{54-50},$$

$$\therefore m = 123456, \qquad e = 54$$

5	4	1	2	3	4	5	6

指数部 e　　仮数部 m

  - 末尾の 78 は記憶できないことに注意

オーバーフローとアンダーフロー

- 指数部 2 桁、仮想部 6 桁で扱える浮動小数点数には限界がある
- 最大値

$$0.999999 \times 10^{+49} \simeq 10^{+49}$$

  - 数値が 10^{+49} を超えると計算が破綻 (オーバーフロー)

- 最小値

$$0.100000 \times 10^{-50} = 10^{-51}$$

  - 数値が 10^{-51} を下回ると計算が破綻 (アンダーフロー)

固定小数点数と浮動小数点数の比較

	固定小数点数	浮動小数点数
例	1234.5678	0.123456×10^4
最大値	$\simeq 10^4$	$\simeq 10^{+49}$
最小値	10^{-4}	10^{-51}
有効桁数	8	6

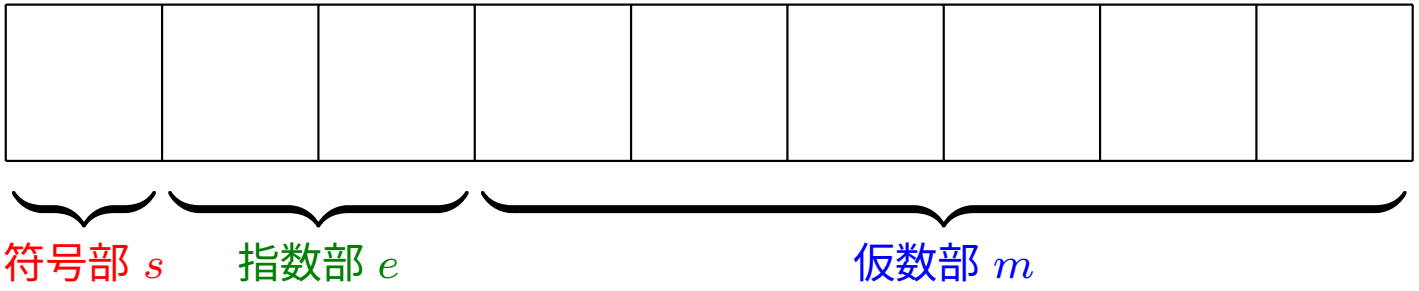
- 浮動小数点数では値の範囲が増加したが、有効桁数が減少

負の数も含めた浮動小数点数

- $(-1)^s = \begin{cases} (-1)^0 = +1 & (s=0) \\ (-1)^1 = -1 & (s=1) \end{cases}$ で正負を表現
- 符号も含めた小数は

$$(-1)^{s} \times 0.m \times 10^{e-50} \qquad (s = 0, 1)$$

- 左端に符号を表現する桁を追加

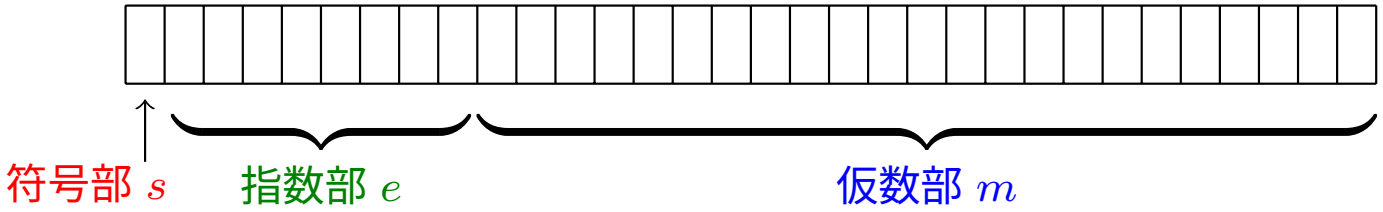


- $s = 0, 1$ なので符号部は 1 ビット (2 進数 1 桁) でよい

現実の浮動小数点数

- 基数の種類
  - 2 進数: 0,1
  - 16 進数: 0,1,2,$\cdots$,9,A,B,C,D,E,F
- 規格の種類
  - IBM 形式
  - IEEE 形式 (i triple e と読む)
- 精度の種類
  - 単精度 (32 ビット)
  - 倍精度 (64 ビット)

IEEE 単精度 (32 ビット) 浮動小数点数



- 符号部は 1 ビットの 2 進数

$$s = 0, 1$$

- 指数部は 8 ビットの 2 進数 (両端 $0, 255$ を含めない理由は後述)

$$0 = (00000000)_2 < e < (11111111)_2 = 255$$

- 仮数部は残りの 23 ビット分だけ 0 or 1 を並べる
  - $(1.m)_2$ と表して小数点以下のみを記憶
  - $(0.m)_2$ と表すと m の一番左が 1 なのは自明 $\rightarrow$ 1 ビット無駄

IEEE 単精度 (32 ビット) 浮動小数点数

- 正規化した浮動小数点数

$$(-1)^s \times (1.m)_2 \times 2^{e-127} \qquad (0 < e < 254)$$

- 正の最大値

$$(-1)^0 \times (\underbrace{1.11\cdots1}_{24\text{ 桁}})_2 \times 2^{254-127} = (2 - 2^{-23}) \times 2^{127}$$

$$= 3.40\cdots \times 10^{38}$$

- 正の “最小値”

$$(-1)^0 \times (\underbrace{1.00\cdots0}_{24\text{ 桁}})_2 \times 2^{1-127} = 2^{-126}$$

$$= 1.17\cdots 10^{-38}$$

正規化した浮動小数点数の最大値

- overflow.c

```
#include <stdio.h>
#include <float.h> // FLT_MAX
#include <math.h>  // nextafterf

int main(){
  // 浮動小数点数の最大値
  printf("%.8e\n", FLT_MAX);

  // FLT_MAX の次の数
  printf("%.8e\n\n",
    nextafterf(FLT_MAX, INFINITY));
}
```

- 実行結果

```
$ gcc overflow1.c -lm
$ ./a.out
3.402823e+38
inf
```

  - 最大値 FLT_MAX は

$$(1.11\cdots1)_2 \times 2^{127}$$

  - FLT_MAX の次は inf

正規化した浮動小数点数の “最小値”

- underflow1.c

```
#include <stdio.h>
#include <float.h> // FLT_MIN
#include <math.h>  // nextafterf

int main(){
  // 正規化した浮動小数点数の最小値
  printf("%.8e\n", FLT_MIN);

  // FLT_MAX の前の数
  printf("%.8e\n\n",
    nextafterf(FLT_MIN, 0.0f));
}
```

- 実行結果

```
$ gcc underflow1.c -lm
$ ./a.out
1.17549435e-38
1.17549421e-38
```

  - 最小値 FLT_MIN は

$$(1.00\cdots0)_2 \times 2^{-126}$$

  - FLT_MIN の前は 0 でない

”最小値”より小さい値

- 2^{-126} より小さい数は非正則数で表現

$$(-1)^s \times (0.m)_2 \times 2^{-126}$$

  - m は 23 個の 0 or 1 の任意の並び
  - m の一番左は 0 でもよい
- 有効桁は 24 桁未満
- 真の (正の) 最小値

$$(-1)^0 \times (\underbrace{0.0\cdots0}_{23\text{ 桁}}1)_2 \times 2^{-126} = 2^{-149}$$

$$= 1.40\cdots \times 10^{-45}$$

浮動小数点数の真の “最小値”

- underflow2.c

```
#include <stdio.h>
#include <float.h>
#include <math.h>

int main(){
  // 正規化した浮動小数点数の最小値
  printf("%.8e\n", FLT_MIN);
  // 浮動小数点数の真の最小値
  printf("%.8e\n", FLT_TRUE_MIN);
  // FLT_TRUE_MIN の前の数
  printf("%.8e\n",
  nextafterf(FLT_TRUE_MIN, 0.0f));
}
```

- 実行結果

```
$ gcc underflow2.c -lm
$ ./a.out
1.17549435e-38
1.40129846e-45
0.00000000e+00
```

  - 真の最小値は

$$2^{-149}$$

  - 真の最小値の前はゼロ

IEEE 単精度浮動小数点数の特別な場合

- $0 < e < 255$ のときは正規化した表式 (再掲)

$$(-1)^s \times (1.m)_2 \times 2^{e-127}$$

- $e = 0$ or 255 のときは特別な数を表す

	$m = 0$	$m \neq 0$
$e = 0$	符号付きゼロ $(-1)^s \times 0$	$(-1)^s \times (0.m)_2 \times 2^{-126}$
$e = 255$	符号付き無限大 $(-1)^s \times \infty$	非数 (Not a number, NaN)

IEEE 単精度浮動小数点数の例

1	1	0	0	0	0	0	1	1	0	1	0	1	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

- 符号部 $s = 1$
- 指数部 $e = (10000011)_2 = 1 \cdot 2^7 + 1 \cdot 2^1 + 1 \cdot 2^0 = 131$
- 仮数部 $(1.m)_2 = (1.010101)_2$
- よって上の IEEE 単精度浮動小数点数が表す値は

$$
\begin{aligned}
(-1)^s \times (1.m)_2 \times 2^{e-127} &= (-1)^1 \times (1.010101)_2 \times 2^{131-127} \\
&= -(1.010101)_2 \times 2^4 \\
&= -(10101.01)_2 \\
&= -(1 \cdot 2^4 + 1 \cdot 2^2 + 1 \cdot 2^0 + 1 \cdot 2^{-2}) \\
&= -21.25
\end{aligned}
$$

IEEE 倍精度 (64 ビット) 浮動小数点数

- 符号部は 1 ビットの 2 進数

$$s = 0, 1$$

- 指数部は 11 ビットの 2 進数

$$0 = (00000000000)_2 < e < (11111111111)_2 = 2047$$

- 仮数部は残りの 23 ビット分だけ 0 or 1 を並べる
  - $(1.m)_2$ と表して小数点以下の 52 ビットのみを記憶
- 正規化

$$(-1)^s \times (1.m)_2 \times 2^{e-1023} \qquad (0 < e < 2047)$$

プログラミングでの整数型と実数型の比較

- 符号付き整数型 (32 ビット)

$$-2147483648 = \underbrace{(100\cdots0)_2}_{32\text{ 桁}} \sim \underbrace{(011\cdots1)_2}_{32\text{ 桁}} = +2147483647$$

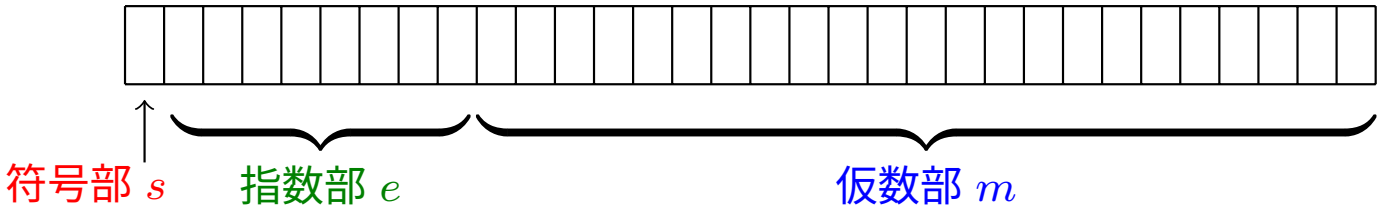
  - 左端の 1 ビットは符号を表す
- 単精度実数型

$$(-1)^s \times (1.m)_2 \times 2^{e-127}$$

	Fortran	C, Java
整数型	`INTEGER I`	`int i;`
単精度実数型	`REAL F`	`float f;`
倍精度実数型	`DOUBLE PRECISION D`	`double d;`

後半まとめ (浮動小数点数)

- 浮動小数点数の構成要素
  - 符号部
  - 指数部
  - 仮数部
- IEEE 単精度浮動小数点数



  - 正規化した表式

$$(-1)^{s} \times (1.m)_2 \times 2^{e-127} \qquad (0 < e < 255)$$

  - $e = 0, 255$ はゼロや無限大、小さな数を表す特別な場合